

Joyory SmartMatch

Smart Shopping Experience for Beauty & Personal Care

Document Type:	Technical Architecture & Development Documentation
Hackathon Track:	Smart Shopping Experience
Date:	September 20, 2026
Authors / Team:	[Student / Developer Team Name] (Editable Placeholder)
Repository:	https://github.com/kavya179/SMART-SHOPPING-EXPERIENCE.git
Core Stack:	React 18 · Django 5 · Django REST Framework · SQLite
Prototype Verification:	26 / 26 Backend Unit Tests Passing (100% OK)

Executive Summary & Abstract

Joyory SmartMatch is a full-stack smart shopping platform built to demystify beauty and personal-care cosmetics. It couples a 5-step guided recommendation quiz with a deterministic weighted scoring engine, side-by-side INCI formulation comparison, an automated zero-cost product FAQ assistant, and a luxury dark obsidian shopping interface. All recommendations and queries are grounded strictly in verified SQLite catalog specifications without external paid AI dependencies.

1. Problem Statement & Market Opportunity

Online beauty and personal care shoppers face significant friction due to formulation complexity and marketing opacity. Key challenges identified across the cosmetic retail space include:

- **Formulation Jargon:** Complex INCI chemical names overwhelm consumers without chemistry expertise.
- **Trial-and-Error Purchasing:** Purchasing incompatible products causes skin irritation, wasted money, and high return rates.
- **Search Limitations:** Standard e-commerce catalogs only support brand or keyword filtering without assessing skin-barrier suitability.
- **Omitted Safety Guidance:** Usage routines (AM vs PM) and essential patch-test instructions are routinely obscured in fine print.

2. Proposed Solution & Concept

Joyory SmartMatch bridges the gap between cosmetic chemistry and consumer decision-making by delivering four core pillars:

1. **Guided 5-Step Quiz Finder:** Interactive workflow assessing category, skin profile, target concern, budget, and active preferences.
2. **Deterministic Scoring Engine:** Pure Python rule-based recommender calculating percentage compatibility with explicit reason breakdowns.
3. **Side-by-Side Comparison Matrix:** Multi-product dock evaluating active concentrations, full INCI formulas, and safety cautions.
4. **Catalog FAQ Assistant:** Automated assistant answering routine and ingredient queries with explicit non-medical guardrails.

3. Target User Personas

Persona	Key Characteristics & Pain Points	Joyory SmartMatch Solution
Skincare Novice	Overwhelmed by active ingredients; fears breakouts.	Guided quiz, beginner active matches, routine directions.
Ingredient Hunter	Seeks specific actives (Vitamin C, Ceramides, SPF).	Full INCI transparency and side-by-side comparison matrix.
Budget Shopper	Strict price cap; seeks value-for-money formulations.	Budget filters, promo codes, and budget-alternative suggestions.

4. Existing Market Review & Proposed Differentiation

Joyory SmartMatch — Development Documentation

Smart Shopping Experience Track

Date of Review: September 20, 2026. An analysis of modern beauty e-commerce platforms highlights the key differentiators introduced by Joyory SmartMatch:

Feature Dimension	Traditional Beauty E-Commerce	Joyory SmartMatch Prototype
Product Discovery	Keyword & brand search only.	5-step category-adaptive compatibility quiz.
Recommendation Logic	Black-box or sponsored placements.	Transparent weighted scoring with explicit reasons.
Formula Comparison	Rarely available or price-only.	Side-by-side INCI, routine, and caution matrix.
Budget Flexibility	Binary filter (hides non-matches).	Suggests closest alternatives with budget explanations.
Catalog Assistance	Generic bot or delayed human tickets.	Instant rule-based FAQ with non-medical guardrails.

5. Feature Status Breakdown

Feature Module	Status	Implementation Mechanism
Product Catalog & Search	Implemented	Django REST API, real-time keyword search & category filter pills.
Product Detail Page (PDP)	Implemented	Full INCI lists, active chips, morning/night routine, cautions.
SmartMatch 5-Step Quiz	Implemented	Category-adaptive step wizard with budget threshold sliders.
Recommendation Engine	Implemented	Deterministic weighted Python scoring with budget fallback logic.
Formula Comparison Studio	Implemented	Floating dock + /compare page for up to 3 selected items.
Wishlist & Cart Management	Implemented	React Context (ShopContext) mirrored to browser localStorage.
Demo Checkout	Simulated	Modal summary, form validation, mock order ID (JOY-XXXXXX).
Product FAQ Assistant	Implemented	Token-based rule engine with non-medical disclaimer banner.
Cloud User Auth	Planned	Multi-device cloud user profiles with JWT authentication.

6. System Architecture

The system adheres to a clean decoupled client-server architecture consisting of three core tiers:

- **Presentation Tier (React 18 SPA):** Client-side routing, glassmorphic UI components, and state management via ShopContext.
- **Application & Logic Tier (Django 5 & DRF):** RESTful API routing, input validation, RuleBasedRecommender service, and ProductFaqAssistant service.
- **Data Tier (SQLite via Django ORM):** Relational storage of product records, ingredient profiles, categories, and routine directions.

```
+-----+ | REACT 18 FRONTEND
(SPA) | | HomePage | Catalog | ProductDetail | SmartMatchQuiz | Compare | Cart | | Global State:
ShopContext (Wishlist, Cart, Compare in localStorage) |
+-----+ | HTTP / JSON REST API
(Port 8000) v +-----+ | DJANGO
REST FRAMEWORK BACKEND | | - Endpoints: /api/products/, /api/recommendations/, /api/faq/ | | -
Services: RuleBasedRecommender, ProductFaqAssistant |
+-----+ | Django ORM Queries v
+-----+ | SQLITE DATABASE | |
Table: products_product (12 Verified Multi-Category Items) |
+-----+
```

7. Database Schema: Product Model

The relational schema is centered around the Product model (backend/products/models.py):

Field Name	Data Type	Constraints / Default	Description
id	AutoField	Primary Key	Unique product ID.
name	CharField(255)	Required	Full product title.
category	CharField(50)	6 Choice Enum	Skincare, Haircare, etc.
price	DecimalField	max_digits=10, dec=2	Base price in INR.
discount_percent	PositiveInt	default=0	Percentage discount.
skin_type	CharField(50)	default='all'	Target skin compatibility.
concern_tags	CharField(255)	Comma-separated	Target skin/hair goals.
key_ingredients	CharField(255)	Required	Primary active ingredients.
full_ingredients	TextField	Required	Full INCI ingredient list.
usage_instructions	TextField	Required	AM/PM routine directions.
caution_info	TextField	Required	Patch test & safety notes.

8. REST API Specification

The backend exposes standard REST endpoints adhering to HTTP status conventions:

Endpoint	Method	Key Parameters	Status & Response Scope
/api/products/	GET	category, search, min_price, max_price, ordering	200 OK: Paginated list of product records.
/api/products/<id>/	GET	id (int)	200 OK: Single product details; 404 if missing.
/api/products/categories/	GET	None	200 OK: Categories with count of items.
/api/recommendations/	POST	category, skin_type, concern, budget_max, preferred_ingredients	200 OK: Ranked matches with match % and explanations.
/api/faq/	POST	question (str), product_id (opt)	200 OK: Catalog answers with non-medical disclaimer.
/api/health/	GET	None	200 OK: Health check status.

9. Recommendation Engine & Matching Rules

The recommendation algorithm (backend/products/services/recommender.py) executes a deterministic weighted scoring formula:

Mathematical Scoring Formula:

Total Score = Category Match (Hard Constraint) + Skin Type Match (30 pts) + Concern Match (35 pts) + Ingredient Boost (20 pts) + Budget Compliance (15 pts)

- **Category Filter:** If product category != user category, product is excluded (Score = 0).
- **Skin Compatibility (30 pts):** 30 pts for exact skin match; 20 pts for universal ('all') formulas; 5 pts otherwise.
- **Concern Alignment (35 pts):** 35 pts if concern in product tags; 20 pts if matched in description.
- **Ingredient Preference (20 pts):** 10 pts per matching preferred active ingredient (up to 20 pts).
- **Budget Compliance (15 pts):** 15 pts if price ≤ budget; linearly scaled down if exceeding budget.
- **Budget Alternative Fallback:** If zero products satisfy budget, closest formulation matches are returned with transparent budget notes.

10. Testing & Quality Assurance

The project has undergone rigorous automated backend unit testing and end-to-end browser user journey validation. All **26 automated unit tests** pass with a 100% pass rate:

Test Suite Module	Test Cases Count	Coverage Scope	Result
ProductModelTests	3 Tests	Model fields, discount calculations, concern list parsing.	3 / 3 PASS
ProductApiTests	6 Tests	Catalog listing, filters, keyword search, price range, categories.	6 / 6 PASS
RecommendationTests	6 Tests	Skincare matches, adaptive haircare, ingredient boost, budget fallbacks.	6 / 6 PASS
FaqAssistantTests	7 Tests	Ingredient lookup, usage directions, cautions, shipping, fallbacks.	7 / 7 PASS
ApiAppHealthTests	4 Tests	Root health checks and CORS response validation.	4 / 4 PASS

11. Limitations & Future Roadmap

Current Prototype Limitations:

- *Simulated Transactions*: Checkout is simulated without real credit card processing.
- *Local Cache Persistence*: Wishlist and cart persist in browser localStorage rather than cloud accounts.
- *Sample Database*: Seeded with 12 multi-category products for demonstration purposes.

Production Roadmap:

- **Phase 1**: Mobile Barcode & INCI Label OCR Scanner using camera input.
- **Phase 2**: AM/PM Daily Routine Tracker with Sunscreen reminders.
- **Phase 3**: Cloud User Profiles & Certified Dermatologist Q&A; integration.
- **Phase 4**: Live Payment Gateway (Razorpay/Stripe) and ERP inventory integration.

12. Conclusion & References

Joyory SmartMatch demonstrates that complex cosmetic shopping can be made delightful, transparent, and safe through rule-based matching algorithms and modern full-stack web engineering.

References:

1. Hackathon Task Brief: Smart Shopping Experience for Beauty & Personal Care (Joyory).
2. Django 5.x & Django REST Framework Official Documentation (<https://docs.djangoproject.com/>).
3. React 18 & Bootstrap 5 Documentation (<https://react.dev/>, <https://getbootstrap.com/>).
4. International Nomenclature of Cosmetic Ingredients (INCI) Standards.